

Phatcharadanai Tangoan

Full Stack Developer — Next.js · React · TypeScript

ABOUT

Computer Engineering graduate (2026), specializing in full-stack web development with a strong frontend focus. Completed a four-month co-operative internship at Trinity Securities, a Thai securities firm, where I was responsible for both the frontend and backend of internal tools. Since graduating I have kept shipping — most recently a deployed, containerized Kanban app that I built and debugged in production end to end.

SKILLS

Languages

TypeScript, JavaScript, Python, SQL, HTML/CSS

Frontend

React, Next.js (App Router), Tailwind CSS, dnd-kit, Framer Motion, Recharts, Leaflet

Backend

Node.js, Next.js API Routes, Prisma ORM, NextAuth, REST APIs, JWT

Databases

PostgreSQL (Neon), MySQL, Redis

Testing

Vitest, GitHub Actions CI

DevOps & Tools

Docker (multi-stage), Git, GitHub Actions, Railway, Vercel, Cloudflare R2

EDUCATION

Mae Fah Luang University

Bachelor of Engineering (Computer Engineering)

Graduated 2026 · GPA 3.45

LANGUAGES

Thai — Native

English — Read technical documentation, English codebases, and issue threads daily.

OTHER

Military service obligation completed (Ror Dor).

EXPERIENCE

Web Developer — Co-operative Internship

Jan – Apr 2026

Trinity Securities Co., Ltd. · Silom, Bangkok

- Built **Tickmatch** end to end — an internal tool that tracks how many times a stock ticks upward during a trading day. Wrote the Python backend that ingests incoming market data files, detects price changes, and pushes updates to the UI, plus the frontend live table with Daily and 1-minute charts. Used **Redis** for the data pipeline and caching, and **Docker** for a reproducible environment.
- Led the frontend of a stock **News Platform** (Next.js + TypeScript) covering watchlists, Premium membership, and real-time Telegram alerts, and built out the initial backend API structure; the AI sentiment-analysis service was owned by a senior engineer on the team.
- Delivered **Reweb Trinity**, a Next.js + TypeScript prototype redesigning the company website for a younger investor audience, built from reusable responsive components.
- Started the internship with no Next.js experience and shipped the first project inside two months; worked in a Git-based workflow alongside the product development team.

PROJECTS

Taskboard — Kanban Collaboration App

May – Jun 2026 · [Live](#) · [GitHub](#)

Next.js 16 · React · Prisma · PostgreSQL · Tailwind v4 · dnd-kit · Docker

- Cut board load from multiple round-trips to a **single JOIN query** by folding a redundant per-request access check into the main query.
- Hardened file uploads to **Cloudflare R2** via presigned URLs: server-side size limits, filename sanitization, MIME allow-list, and cleanup of orphaned objects when a card is deleted.
- Auth with credentials + OTP email verification and Google OAuth (**NextAuth**); viewer/editor roles enforced server-side. Sign-up is atomic — a Prisma **\$transaction** commits the user and its one-time code together or rolls both back.
- Drag-and-drop with optimistic UI, checklists, comments, labels, priorities, due dates; accessibility pass for **aria-modal** dialogs and touch targets, plus a fix for a leaked ESC key listener.
- Shipped in a multi-stage **Docker** build (Alpine, non-root, Next.js standalone) to Railway with Neon PostgreSQL — including debugging a Prisma musl/OpenSSL engine failure and a production login redirect loop — and guarded by a **Vitest** suite (28 tests) plus **GitHub Actions CI** (lint, typecheck, test).

Suicide Risk Surveillance System — Senior Project

[GitHub](#)

Next.js · PostgreSQL · Prisma · Python (scikit-learn) · Leaflet · Recharts

- Team project — a surveillance dashboard for Chiang Rai built in **Next.js (App Router)** + TypeScript on PostgreSQL with Prisma, organised into typed API routes and reusable components. A teammate owned the Random Forest model; I worked on the web application.
- Built the ingestion pipeline: CSV upload triggers a Python script that engineers features (income banding, age grouping, season), applies the saved encoders, runs prediction, and writes results into PostgreSQL — so staff refresh the dashboard without touching a database.
- Built the analytics layer — a **Leaflet**/GeoJSON choropleth shading each district by risk tier with drill-down to subdistricts, plus filterable Recharts line, bar, and pie breakdowns.